EDAM: Edit Distance tolerant Approximate Matching content addressable memory

Robert Hanhan[1], Esteban Garzón[2,3], Zuher Jahshan[1], Adam Teman[2], Marco Lanuzza[3], and Leonid Yavits[2]

[1] Department of Electrical Engineering, Technion-Israel Institute of Technology, Haifa 3547902, Israel
[2] EnICS Labs, Faculty of Engineering, Bar-Ilan University, Ramat-Gan 5290002, Israel
[3] Department of Computer Engineering, Modeling, Electronics and Systems, University of Calabria, Rende 87036, Italy
{roberthanhan,zuherjahshan}@campus.technion.ac.il; esteban.garzon@unical.it; adam.teman@biu.ac.il; m.lanuzza@dimes.unical.it; leonid.yavits@nububbles.com

ABSTRACT

We propose a novel edit distance-tolerant content addressable memory (EDAM) for energy-efficient approximate search applications. Unlike state-of-the-art approximate search solutions that tolerate certain Hamming distance between the query pattern and the stored data, EDAM tolerates edit distance, which makes it especially efficient in applications such as text processing and genome analysis. EDAM was designed using a commercial 65 nm 1.2 V CMOS technology and evaluated through extensive Monte Carlo simulations, while considering different process corners. Simulation results show that EDAM can achieve robust approximate search operation with a wide range of edit distance threshold levels. EDAM is functionally evaluated as a pathogen DNA detection and classification accelerator. EDAM achieves up to 1.7× higher F_1 score for high-quality DNA reads and up to 19.55× higher F_1 score for DNA reads with 15% error rate, compared to state-of-the-art DNA classification tool Kraken2. Simulated at 667 MHz, EDAM provides 1,214× average speedup over Kraken2. This makes EDAM suitable for hardware acceleration of genomic surveillance of outbreaks, such as the ongoing Covid-19 pandemic.

1 INTRODUCTION

Content-addressable memories (CAM) are widely used in microprocessors (fully associative cache memories, translation lookaside buffers, branch prediction buffers, superscalar renaming circuitry and so on), as well as in network routers and switches, and other domain specific hardware [9, 16]. CAMs can also be employed to speed up big data applications: for example, 22 out of 37 BigDataBench–3.2 [25] workloads use pattern search [46].

While CAMs are typically designed to find exact matches, approximate or similarity search is increasingly required by many popular contemporary data-intensive applications, such as data analytics, machine learning, deep learning, and computational biology [3, 31, 35–37]. Bioinformatics, in general, and genome analysis, in particular, have gained keen interest of the research community due to the exponential growth of the sequenced DNA data volume in recent years [69]. Genome analysis is used in a wide variety of applications, from personalized healthcare, through monitoring environmental ecosystems and genomic surveillance (being deployed worldwide to fight the Covid-19 pandemic), to sustainable agriculture [68].

In approximate search, if the difference between a stored pattern and the query pattern is below a certain predefined threshold, such stored pattern is considered a "match". While in some cases, the difference is limited to replacements (i.e., where a data element is replaced by another), in applications involving text and sequenced DNA data, there are two additional types of difference: insertions (where a data element is inserted into an otherwise identical sequence of data elements), and deletions (where a data element is deleted from a sequence), collectively called indels. Replacements, insertions and deletions are typically referred to as edits.

A variety of CAM designs support approximate search. These solutions target Hamming distance tolerance, typically limited to only a few bits [32, 61]. However, a single insertion or deletion (indel) can result in a very large Hamming distance, as demonstrated by Fig. 1(a). Such a single edit may comprise tens of bits in Hamming distance, which would almost certainly be above the tolerance threshold of state-of-the-art approximate search CAMs.

To alleviate this fundamental limitation, we propose a novel edit distance-tolerant content addressable memory (EDAM), which is capable of tolerating a user-configurable edit (rather than Hamming) distance. EDAM targets the high speed approximate search in applications such as text or genomic processing and analysis, where edits include indels in addition to simple replacements.

Traditionally, pathogen detection relies upon the identification of pre-established markers of a particular disease [66]. However, pathogen detection, as well as DNA classification in general, is increasingly accomplished by sequencing of metagenomic samples [8]. In particular, DNA classification is performed as follows: (1) a sample potentially containing DNA of multiple organisms is obtained and prepared; (2) the sample is sequenced; the sequencer outputs DNA reads (potentially sourced from different organisms); DNA sequencers are prone to sequencing errors, such as indels and replacements; (3) DNA reads (soiled with sequencing errors) are processed by a metagenomic classification application that potentially associates each DNA read with a certain species in its database. A variety of DNA classifiers have been proposed: Phymm and PhymmBL [11], CLARK [57], CLARK-S [56], Kraken [72] and Kraken2 [71]. In particular, state-of-the-art DNA classifiers, such

ISCA '22, June 18–22, 2022, New York, NY, USA

ACM ISBN 978-1-4503-8610-4/22/06…$15.00
https://doi.org/10.1145/3470496.3527424

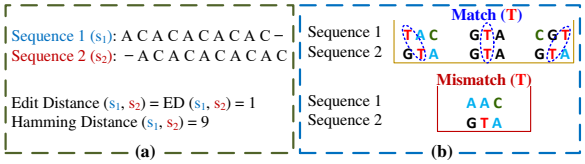

Figure 1: (a) Edit vs. Hamming distance, (b) EDAM edit distance estimation: In addition to the co-located data element, it also evaluates its left and right neighbors.

as Kraken, classify DNA by exact matching of DNA read fragments (called k-mers) against an existing DNA database, e.g., a collection of pathogen DNA. Since DNA reads typically contain sequencing errors, a certain fraction of query k-mers would not hit in the database, thus limiting the sensitivity of conventional DNA classifiers.

The operating principle of EDAM is based on the observation that an insertion or deletion shifts a part of the data pattern right or left, as shown in Fig. 1. Hence, by matching not only the co-located data elements but also their left and right neighbors, it is possible to tolerate insertions and deletions, as shown in Fig. 1(b). If none of the three candidate data elements (co-located, left, and right neighbors) matches, a single element mismatch occurs. If the number of such mismatches is below a certain configurable edit distance threshold, we consider it a match. Conversely, if the number of single mismatches exceeds the edit distance threshold in every EDAM row, it means that the query pattern misses in EDAM.

Our paper makes the following main contributions:

- To the best of our knowledge, EDAM is the first approximate search CAM design that can tolerate large user-programmable edit distance.
- EDAM employs matchline charge redistribution as a measure of edit distance, and does not require data transformation, such as error correction codes or locality-sensitive hashing;
- EDAM tolerates and differentiates between patterns with large edit distances with very high sensitivity and precision;
- EDAM is fully evaluated exploiting a commercial process technology, covering all local variations around TT, SS, and FF corners, as well as variations in design parameters.

The rest of the paper is organized as follows: Section 2 discusses the background and motivation of our work; Section 3 presents the edit distance tolerant approximate matching CAM design; Section 4 explores EDAM design space, while Section 5 details EDAM application to virus DNA classification, and discusses the results. Finally, Section 6 concludes this work.

2 BACKGROUND & MOTIVATION

2.1 Content-Addressable Memory

Fig. 2 shows the architecture of a conventional content-addressable memory (CAM) array of n columns and m rows [59]. A CAM compares between the query data held in the search data registers, and the information typically stored in six-transistor static random access memory (6T-SRAM) bitcells. A matchline (ML) is shared between bitcells of an n-bit word and also fed into a sense amplifier (SA). Searchlines (SLs), denoted SL and $\overline{\text{SL}}$, are shared by all CAM rows. CAM read and write operations are accomplished in the same way as in the conventional 6T-SRAM. To access a bitcell,

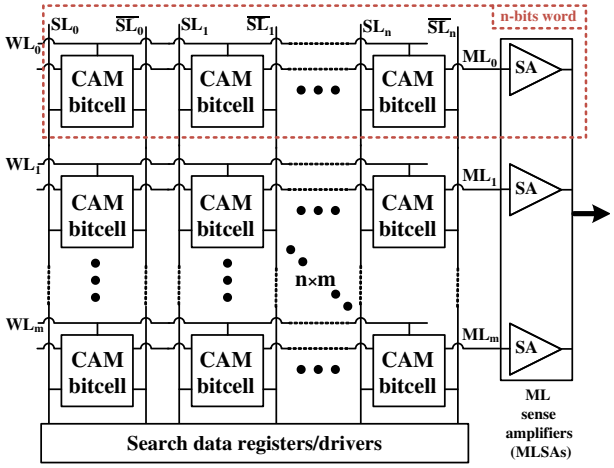

Figure 2: Conventional content-addressable memory (CAM) array.

the word line (WL) enables the row, and the SLs are precharged for read or asserted (complementary values) for write. A search (compare) operation is performed simultaneously throughout the entire array within a single clock cycle by asserting the query data on the searchlines. The matchline sense amplifiers (MLSAs) sense the state of the matchlines at the end of the compare cycle and signal match or mismatch.

2.2 Approximate Search Content-Addressable Memory

Many ternary and binary NOR- and NAND-based CAM bitcell designs have been proposed in recent years, including both CMOS-based [2, 6, 14, 15, 17, 20–22, 29, 33, 50, 60, 67, 75, 76] and emerging-memory-based [34, 36, 62, 73, 74] solutions. Several CAM designs offer soft-error tolerance using error correction coding (which requires memory redundancy) and replacing the matchline sense amplifier with an analog comparator [41, 58]. Such designs typically only tolerate a limited Hamming distance (1–4 bits). Another class of approximate search CAMs uses locality sensitive hashing of stored data and query patterns [53, 63]. While such schemes potentially tolerate large Hamming distances, they require hashing of data prior to storage and search. Additionally, large Hamming distance does not always result in low similarity of hashed data sketches [48], which leads to false positive results and hence limited precision. A CAM for minimum Hamming distance search that uses digital circuitry for bit comparison, as well as winner-take-all functionality, is proposed in [49].

Several emerging memory (memristor crossbar) based designs for Hamming distance approximation have also been proposed [70, 77]. NCAM [13] uses near-memory logic to calculate the sum of squares of data word differences, which measures the similarity between data vectors. PPAC [19] calculates Hamming similarity by performing a population count, by tallying the number of ones over all XNOR outputs of the CAM bitcells of a data word.

Some of the approximate search CAM designs use timing (i.e., the matching score signal delay, or the speed of the matchline discharge) as a measure of Hamming distance. HD-CAM [27], a Hamming distance tolerant CAM, uses the combination of the voltage, controlling the speed of the matchline discharge, and the

sense amplifier reference voltage to define the Hamming distance threshold. HD-CAM is capable of tolerating very large Hamming distances. However it does not directly support the edit distance tolerance. A Hamming distance search CAM, where the matching score signal is delayed every time a bit mismatch occurs, is proposed in [12]. In the approximate search enabled CAM for energy efficient GPUs, proposed in [61], a small Hamming distance ($\leq$ 2 bits) is tolerated through meticulous timing of the matchline discharge. In [32], Hamming distance of up to 4 bits is tolerated by using delay lines at the clock inputs of four separate sense amplifiers on each matchline. Tunable sampling time techniques require very precise device and circuit sizing, while achieving limited sensitivity and precision (due to false mismatches and multiple false matches [61]). These issues are exacerbated by process variations.

2.3 DNA Classification

DNA is composed of four nucleotides: Adenine (A), Guanine (G), Cytosine (C), and Thymine (T), which are frequently referred to as DNA basepairs, bases or bps. Accordingly, a DNA data element is a DNA base that can have one of four values (A, G, C and T). DNA sequencing is the process of determining the bases of a DNA chain. Contemporary high-throughput DNA sequencers can sequence multiple DNA samples in parallel [30]. A DNA sequencing process, along with the genomic analysis, is carried out in several steps [39]: (1) sample preparation; (2) DNA sequencing that generates multiple DNA reads; and (3) DNA classification, DNA read alignment, genome assembly, variant analysis, etc.

DNA read alignment is one of the most time consuming parts of the genome analysis pipeline[64]. A variety of read alignment solutions [4, 42–45], as well as read alignment hardware accelerators exist. The optimal sequence alignment, such as dynamic programming based Smith-Waterman, is a central step of read alignment algorithms. Its complexity is $O(s_1 \cdot s_2)$ where s_1 and s_2 are the lengths of the sequences.

The goal of DNA classifier is to find what organism a DNA sequence (or a single DNA read) belongs to, taking into account that it possibly originates from a metagenomic sample (i.e., a sample that contains DNA of many different species). Several probabilistic classifiers have been proposed, such as interpolated Markov model based Phymm and PhymmBL [11], BLAST-based models [4], MEGAN [23] and METAPHYLER [47], naive Bayesian classifier NBC [65], and others. These classification tools are sensitive but relatively slow. For example, DNA classification using Smith-Waterman like dynamic programming would have the complexity ranging from $O(m \cdot n^2)$ (the best case) to $O(m^2 \cdot n^2)$ (the worst case), where m is the number of DNA reads and n is the read length.

Recently, PACIFIC, a CNN based SARS-CoV-2 classification solution has been proposed [1]. SquiggleFilter [24] is a virus detection framework that directly analyzes ONT MinION [54] raw squiggles and filters out all but the target virus DNA reads.

To speed the DNA classification up, exact pattern matching classifiers were developed, including CLARK [57], CLARK-S [56], Kraken [72] and Kraken2 [71]. These classifiers exhibit significantly higher speed at the cost of limited sensitivity. One reason for the decline in sensitivity is sequencing errors that are inherently present in DNA reads. These sequencing errors manifest in replacing bases

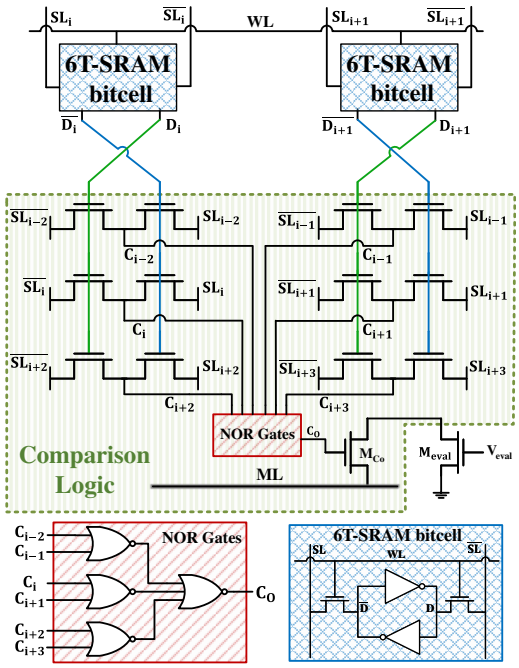

Figure 3: Edit Distance tolerant Approximate Matching CAM (EDAM) cell. Note that the cell comprises two SRAM bitcells, storing a single DNA base.

in DNA reads with incorrect ones, deleting bases, or inserting redundant bases. As a result, DNA read fragments (k-mers) that otherwise should have matched in the classification database, end up being unclassified and discarded.

Approximate search CAMs alleviate this fundamental flaw. When comparing a sequence s_1 against a set of sequences S contained in an approximate search CAM, every $s_2 \in S$ such that *Hamming Distance* $(s_1, s_2) \leq t$ for some threshold t, will match. However, as discussed above, DNA reads contain indels. A single indel may result in a very significant Hamming distance, as shown in Fig. 1(a). To endure such sequencing errors, the tolerance threshold t needs to be relatively high. This increases the sensitivity of classification, but at the same time, significantly reduces its precision (by allowing a high false positive rate).

In this work, we present a fast, highly sensitive and precise approximate matching-based DNA detection and classification solution, implemented by EDAM.

3 EDIT DISTANCE TOLERANT APPROXIMATE MATCHING CAM DESIGN

An example of EDAM operation is shown in Fig. 1(b), vis-a-vis a single DNA base (T in this example). In the top scenario, sequence 1 is either exactly aligned with sequence 2 (in the middle), or is shifted left or right (due to an indel). In both latter cases, the edit is tolerated, and the result of the per-base compare is a match. In the bottom scenario, none of the relevant bases of sequence 1 (left, co-located, or right) matches T, hence the per-base compare result is a mismatch. If the number of such mismatches (typically caused by replacements or consecutive indels) exceeds a certain edit

distance threshold, the result of a comparison between sequence 1 and sequence 2 is a mismatch.

In bioinformatics applications, such as DNA classification, a single DNA base can be encoded using two bits. Fig. 3 shows the schematic of the EDAM cell, tailored for bioinformatics applications, comprising two conventional 6T-SRAM cells (used to store a single base), and the comparison logic circuit along with an evaluation transistor (M_{eval}). The ML discharge rate is set by evaluation voltage (V_{eval}), which controls the conductivity of the M_{eval} transistor. This design enables exact and approximate search operations by asserting $V_{eval} = V_{DD}$ or $0 < V_{eval} < V_{DD}$, respectively.

3.1 EDAM array design

Similar to conventional CAMs [59], the query pattern is driven onto the searchlines and compared with the data stored within the EDAM array. While in conventional CAMs, the search is limited to comparing the bit-colocated query with the stored data, in EDAM, the comparison window is extended to the neighboring elements of the query pattern. The inputs to the comparison logic circuit (Fig. 3) are three neighboring searchlines and their complements (SL_{i-2} through SL_{i+3} and $\overline{SL_{i-2}}$ through $\overline{SL_{i+3}}$), which are compared with the D_i and D_{i+1} bits that are stored in the 6T-SRAM bitcells, representing a DNA base.

Bit (D_i) (in position i) is compared with the query pattern bit i (SL_i), and its left and right neighbors in positions $i-2$ (SL_{i-2}) and $i+2$ (SL_{i+2}), respectively. Likewise, bit (D_{i+1}) (in position $i+1$) is compared with the query pattern bit i+1 (SL_{i+1}) and its left and right neighbors, SL_{i-1} and SL_{i+3} in positions $i-1$ and $1+3$, respectively. Each pair of nMOS transistors connected in series (see comparison logic block in Fig. 3) is logically equivalent to a two-input XOR gate with D_i and SL_i as its inputs and C_i as its output. The outputs (C_{i-2} up to C_{i+3}) of the equivalent XOR gates are aggregated by the NOR gates block in Fig. 3, whose final output, C_O, controls the M_{Co} transistor.

EDAM utilizes the observation that if every per-base mismatch creates a certain electrical charge reduction on the ML, then the total ML voltage drop is proportional to the edit distance between a query pattern and the pattern stored in a CAM row. EDAM exploits such a voltage drop as a measure of edit distance.

When the two-bit data element (coding a DNA base) stored in 6T-SRAM bitcells matches at least one of the (left, center, right) query data elements (bases), i.e., if D_i-$D_{i+1} = SL_{i-2}$-SL_{i-1} or SL_i-SL_{i+1} or SL_{i+2}-SL_{i+3}, the output C_O of the NOR gates block is zero. This turns off the M_{Co} transistor, preventing the ML from discharging through the EDAM cell. When the two-bit data element stored in EDAM matches none of the three query data elements (i.e., D_i-$D_{i+1} \neq SL_{i-2}$-SL_{i-1} and SL_i-SL_{i+1} and SL_{i+2}-SL_{i+3}), the C_O outputs logic '1', signaling a per-base mismatch. In this case, M_{Co} is turned on, creating a discharge path from ML through M_{eval} of the EDAM cell. The speed of ML discharge is set by the V_{eval}.

An EDAM row, along with its precharge and SA circuitry (designated an EDAM word), is shown in Fig. 4(a). Precharge ($\overline{PC\text{-}ML}$) and predischarge (PdC-SL) signals are used to precharge and predischarge ML and the SLs, respectively. Note that the left and right SLs associated with the left and right query bases are input to each EDAM cell (refer to Fig. 3).

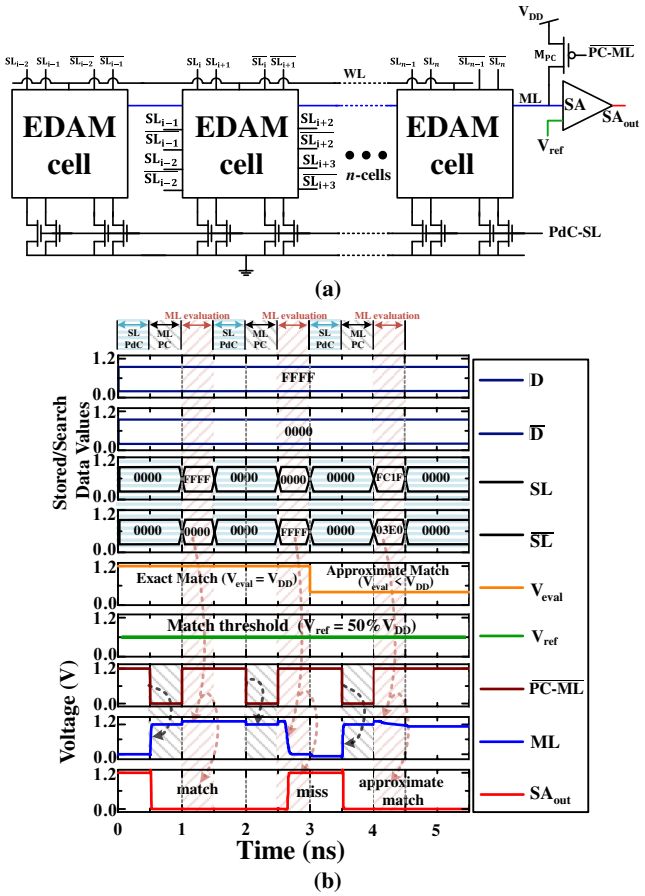

Figure 4: (a) EDAM n-cell word with a conventional precharge circuitry ($\overline{PC\text{-}ML}$ for matchline precharge and PdC-SL for SL predischarge) and a sense amplifier (SA). (b) Timing diagram of EDAM (a single 16-bit EDAM word). The stored and query patterns are presented as hexadecimal numbers. The nominal voltage is V_{DD} = 1.2 V.

3.2 Timing

Fig. 4(b) shows the timing diagram of several compare operations (in a single 16-bit EDAM word) under the nominal voltage (V_{DD} = 1.2 V). Write and read operations are trivial (similar to write and read in SRAM), and therefore, are not shown. A compare operation comprises two steps, precharge and evaluation. In the precharge step, the SLs are discharged and then the ML is precharged to V_{DD} by opening the M_{PC} transistor ($\overline{PC\text{-}ML}$ = '0'). In the evaluation step, the M_{PC} transistor is disabled and the query data is driven onto the SLs. The edit distance threshold is set by a combination of the evaluation voltage V_{eval} and the SA reference voltage V_{ref}. The compare result ('0' for match and '1' for mismatch) is signaled at the SA output.

Fig. 4(b) presents three consecutive compare operations with precharge and evaluation steps taking 1 ns and 0.5 ns, respectively. The first two (0–3 ns timeframe) are exact searches with the first one resulting in a match (the edit distance between the query and the stored pattern is zero) and the second resulting in a mismatch (the edit distance is above zero). The third compare operation (3–4.5 ns timeframe in Fig. 4(b)) is an approximate search, resulting in a match (the edit distance is below the threshold level).

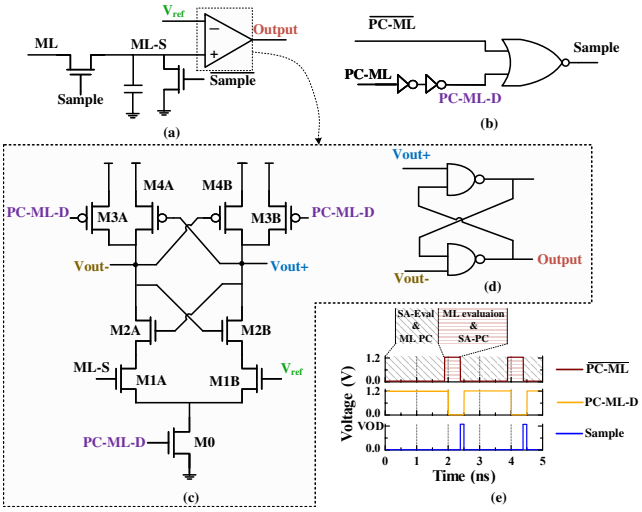

Figure 5: Matchline Sense Amplifier schematics and waveforms.

To enable the exact search operations, M_{eval} is fully open ($V_{\text{eval}} = V_{\text{DD}}$). The approximate matching operation depends on the conductivity of the M_{eval} transistor. During the approximate search, V_{eval} is below the nominal voltage ($V_{\text{eval}} < V_{\text{DD}}$), limiting the conductivity of M_{eval}.

3.3 Sense amplifier design

The matchline sense amplifier (MLSA) is based on a classical StrongARM design [40]. The ML sensing design and circuitry is shown in Fig. 5, as follows: a Sample and Hold circuit (S&H) (Fig. 5(a)), a timing unit (Fig. 5(b)), a cross-coupled amplifier (Fig. 5(c)), which in an open loop configuration works as a comparator, and an RS latch (Fig. 5(d)). While ML is being evaluated, the cross-coupled amplifier is in a PC phase dictated by transistors M3A and M3B, controlled by a delayed PC-ML (i.e., PC-ML-D), where the outputs are charged to V_{DD}. Upon completion of the evaluation phase, a pulse (Sample) triggers the charging of a small capacitor to the ML voltage, labeled ML-Sampled (ML-S). The comparator evaluates the ML-S compared to an evaluation threshold (V_{ref}), which is the tolerance reference voltage, yielding a match or mismatch when the ML voltage is above or below V_{ref}, respectively. The result is fed into the RS latch, which compensates for metastable behavior in the MLSA PC phase. Sampling the value of ML onto a capacitor frees the mutual dependence of the evaluation time of the MLSA response and the PC of the ML for the next evaluation. This allows the creation of two interlocked cycles – while ML is precharged, the S&H result is being evaluated, and while the SA is being precharged, the ML is evaluated. This results in zero-downtime of the system, and every PC cycle has a corresponding evaluation cycle done in parallel, as shown in Fig. 5(e). In addition to the sampling device, an inverse device exists in order to discharge the ML-S node for the next sample.

3.4 Consecutive indels mitigation

In the exact search mode, EDAM efficiently tolerates insertions and deletions of one or multiple *single* data elements (DNA bases). However, if an indel comprises several consecutive data elements, such indel may cause a mismatch and in turn limit the sensitivity.

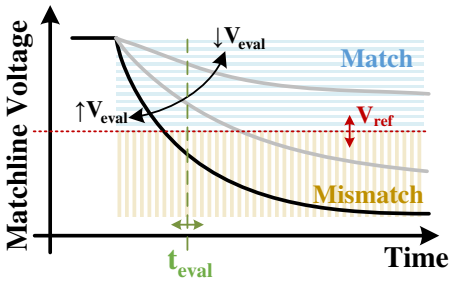

Figure 6: Match / mismatch as a function of the design space variables: evaluation voltage (V_{eval}), SA reference voltage (V_{ref}), and evaluation time (t_{eval})

One way to mitigate this drawback is to perform several consecutive queries (searches) with the same query pattern shifted left and right. For example, if there is a two-data element insertion in the query, there is a two-element shift between the query and a part of the stored pattern, leading to a mismatch in the exact search mode. However, if we shift the query pattern left by one data element and repeat the search, it will return a match, since the query data elements left of the insertion become one position apart from the stored pattern, and will match as the left neighbors, while the data elements right of the insertion also become one position apart from the stored pattern, and will match as the right neighbors. By performing multiple queries with a shifted query pattern, we can improve the sensitivity at the cost of a longer search time.

4 DESIGN SPACE EXPLORATION

The outcome of the ML evaluation during the approximate search, i.e. a match or a mismatch, is defined by the following three design space variables: (1) the ML evaluation time (t_{eval}) (refer to Fig. 4); (2) the voltage that controls the conductivity of the M_{eval} transistor (V_{eval}) (refer to Fig. 3); (3) the SA reference voltage (V_{ref}) that sets the match threshold. Fig. 6 presents the EDAM design space in a nutshell.

We use *Sensitivity* and *Specificity* to evaluate the approximate search results, as follows:

$$Sensitivity = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad Specificity = \frac{\text{TN}}{\text{TN} + \text{FP}} \tag{1}$$

where TP and FN are true positive and false negative results, and TN and FP are true negative and false positive results, respectively.

A true positive (TP) result, or a true match, is obtained when a comparison of two patterns with edit distance **below** the edit distance threshold results in a **match**; a false negative (FN) result, or a false mismatch, is obtained when such comparison results in a **mismatch**.

A true negative (TN) result, or a true mismatch, is obtained when a comparison of two patterns with edit distance **above** the edit distance threshold results in a **mismatch**; a false positive (FP) result, or a false match, is obtained when such comparison results in a **match**.

To evaluate the susceptibility of EDAM approximate search results to the variations in V_{eval}, V_{ref} and t_{eval}, we employ extensive MC simulations, carried out at the circuit-level using a commercial 65 nm 1.2 V CMOS technology. Due to the device process variations, EDAM may require adjusting the V_{eval} and V_{ref} levels to maintain

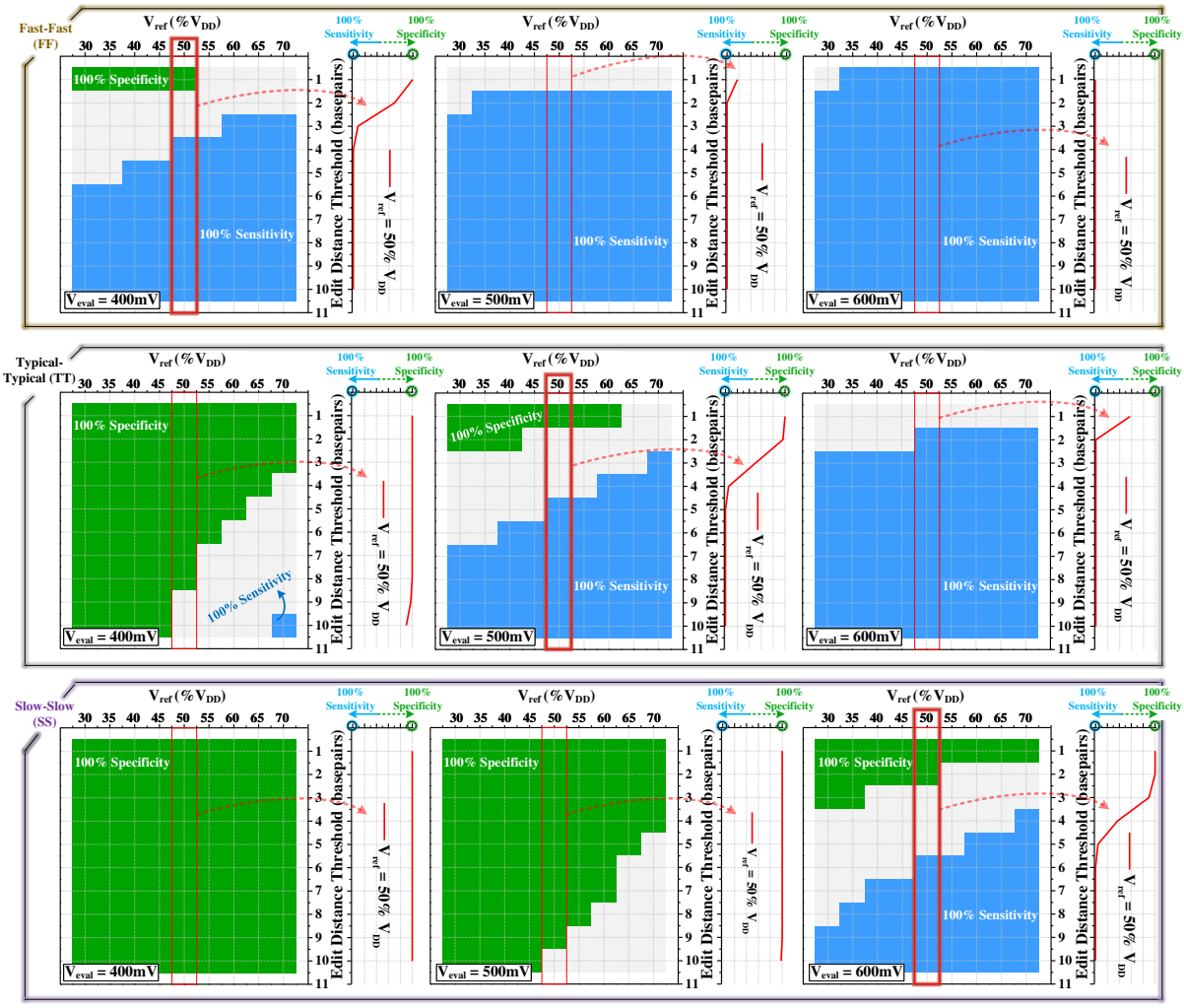

Figure 7: EDAM susceptibility to local variations for Fast-Fast (FF), Typical-Typical (TT), and Slow-Slow (SS) corners, based on exhaustive Monte Carlo simulations. The green and blue colors of the colormap emphasize 100% specificity and sensitivity (true results only) regions, respectively. Red boxes along the diagonal indicate how adjusting the V_{eval} and V_{ref} allows maintaining 100% sensitivity and specificity for a given edit distance threshold in different process corners. The evaluation time (t_{eval}) is 1.0 ns.

the edit distance threshold, i.e. to keep the ML voltage above or below the V_{ref} to produce a match or mismatch, respectively. Fig. 7 shows the EDAM susceptibility to local variations for Fast-Fast (FF), Typical-Typical (TT), and Slow-Slow (SS) corners. Sensitivity and specificity are plotted as functions of V_{ref} for different levels of V_{eval} and a fixed value of t_{eval}. The areas with 100% specificity and sensitivity (100% true results) are colored green and blue, respectively. The median areas, where the specificity and sensitivity are below 100%, are transition regions, where false match and mismatch may occur. Fig. 7 shows that our design exhibits very high sensitivity and/or specificity in all design corners. To the right of each sub-plot, a 2-D representation of the sensitivity/specificity as a function of the edit distance threshold is provided. These plots correspond to the area of the highlighted red boxes in the plot to their left, where $V_{ref} = 50\%V_{DD}$.

Consider the following example. Suppose a certain application requires an edit distance threshold of 1 with 100% specificity. For the TT corner, this requirement is satisfied by setting V_{eval} at 500 mV and V_{ref} at $50\%V_{DD}$, as highlighted by the red box in the center sub-plot of Fig. 7. The 2-D representation of this scenario is shown in the curve profile to the right of the colormap, emphasizing that the requirement is met. However, if the EDAM chip targeted for this application is manufactured in the FF or SS corner, the required edit distance threshold level can be obtained by respectively reducing (to 400 mV) or increasing the V_{eval} (to 600 mV) applied at the gate of M_{eval}.

In general, edit distance threshold level can be maintained by adjusting the values of t_{eval}, V_{ref}, and V_{eval}, with each such (t_{eval}, V_{ref}, V_{eval}) combination corresponding to a specific threshold level.

To examine the effects of t_{eval} and V_{ref} on the sensitivity and specificity of EDAM approximate search, we have evaluated a 128-bit (64 cells) EDAM operating at V_{eval} of 500 mV at the TT-corner with the edit distance threshold of 3. Fig. 8 shows the impact of t_{eval} and V_{ref} on the specificity and sensitivity. The areas with 100%

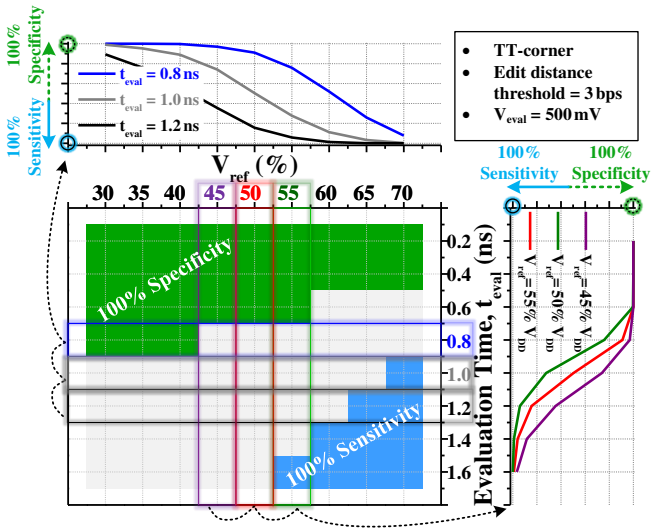

Figure 8: The effect of t_{eval} and V_{ref} on specificity and sensitivity. V_{eval} =500 mV; Edit distance threshold is 3.

specificity and sensitivity (100% true results) are colored green and blue, respectively. Blank is the transition region where false results are possible. The top and right-side projections of the colormap of Fig. 8 plot the sensitivity/specificity for fixed values of t_{eval} and V_{ref}, respectively. For example, to ensure that the mismatch is always true (100% specificity) for any value of V_{ref} in the 30% V_{DD} to 70% V_{DD} range, t_{eval} must be less than 0.6 ns. However, the same result can be also obtained by reducing the M_{eval} transistor conductivity, by tuning the V_{eval} down.

5 APPLICATION & RESULTS

Testing by DNA sequencing and genomic analysis can be used to diagnose viral diseases, augmenting or potentially replacing the popular PCR testing [7, 10, 26]. Genomics surveillance, which is gaining momentum in light of the ongoing Covid-19 pandemic, is a necessary tool for tracing the spread of virus mutations and variants, and is critical for governments to establish proper protocols and to help controlling the pandemic [5, 55]. A cost-effective, fast and reliable (i.e., highly sensitive and precise) solution for virus DNA (and pathogen in general) detection and classification is critical to enabling the worldwide genomic surveillance [7, 10].

5.1 EDAM as DNA classifier

Virus DNA classification, whose aim is to overcome the shortcomings of existing testing techniques [7, 26], can strongly benefit from the approximate matching capabilities of the proposed EDAM.

Fig. 9 presents an EDAM-based virus detection accelerator, which employs the edit distance tolerant approximate matching as a component of a virus DNA classification-by-sequencing pipeline. The DNA of the target virus (designated as the reference DNA) is known in advance and represented by a set of short DNA fragments (k basepairs long), referred to as k-mers [52]. The reference DNA k-mers are extracted, as follows (refer to Fig. 9(b)): the first k-mer is the reference DNA fragment from position 0 to position $k-1$, the second k-mer is the reference DNA fragment from position 1 to position k, and so on. The reference DNA database is generated by storing k-mers in the EDAM offline, prior to the classification operation, where a unique k-mer is stored in a separate EDAM row, as shown in Fig. 9(b). The number of k-mers in the EDAM is bounded by $N-k+1$, where N is the length of the virus DNA. In the case of Severe Acute Respiratory Syndrome coronavirus-2 (SARS-CoV-2), N is approximately 29 thousand, where some k-mers may appear more than once. Therefore, the actual number of k-mers in the EDAM could be lower. In our evaluation, we use k=64 (a 64-mer).

A sequenced sample typically consists of a large set of DNA reads, sourced from DNA of different organisms (e.g., bacteria and viruses) presented in the sample. The set of DNA reads in the FASTQ [18], or similar, format is assumed to be placed in the system memory. The virus detector (Fig. 9(a)) fetches the DNA reads to the read buffer that feeds the shift register. The memory bandwidth required to support the peak EDAM throughput is 10.8GB/s. Each new DNA read is loaded from the read buffer into the shift register. A segment of the shift register, 64 basepair (128 bit)-wide, feeds the EDAM array. During the detection operation, the read is shifted right one basepair (two bits) per cycle in a sliding window fashion. As a result, a single 64-mer is searched in EDAM in every clock cycle. The process is controlled by a simple micro-controller implemented by a state machine; its control registers are memory mapped to allow the host access.

Ideally, k-mers that belong to the target virus DNA, should match exactly in the EDAM. However, DNA reads contain sequencing errors of replacement, insertion and deletion types [38]. Another source of difference between the query DNA reads and the reference DNA are genetic variations, which may occur in mutations, such as alpha or delta variants of SARS-CoV-2. Such variations result in a nonzero edit distance between a read and a reference fragment (k-mer) that would otherwise match exactly.

The ability of EDAM to tolerate edit distance as well as large Hamming distance enables DNA classification, even when the target DNA reads have multiple sequencing errors or genetic variations. Moreover, by allowing the programmable edit distance threshold, EDAM supports a wide variety of sequencing error profiles.

The reads sourced from DNAs of other organisms presented in the sample are expected to exhibit significant difference vs. the target virus DNA, such that the edit distance between those reads and the reference k-mers should typically be higher than the edit distance threshold, which in turn allows classifying those reads as not SARS-CoV-2.

If a 64-mer approximately matches anywhere in EDAM, the hit counter $\sum Hits$ (whose input is the OR of all MLSA outputs, see Fig. 9(a)) is incremented. After the DNA read is processed, the hit counter is compared with a user-defined hit threshold; if it exceeds the hit threshold value, the DNA read is classified as SARS-CoV-2. The result is written to the system memory.

To enable virus classification in addition to detection, EDAM needs to retain a database of different virus genomes as opposed to a single genome. An encoder is added to the MLSA outputs of EDAM, signalling the number of the matching row, which is then used to determine which reference genome the k-mer matched in.

EDAM edit distance threshold can be dynamically calibrated using a validation set comprising synthetic DNA reads or DNA

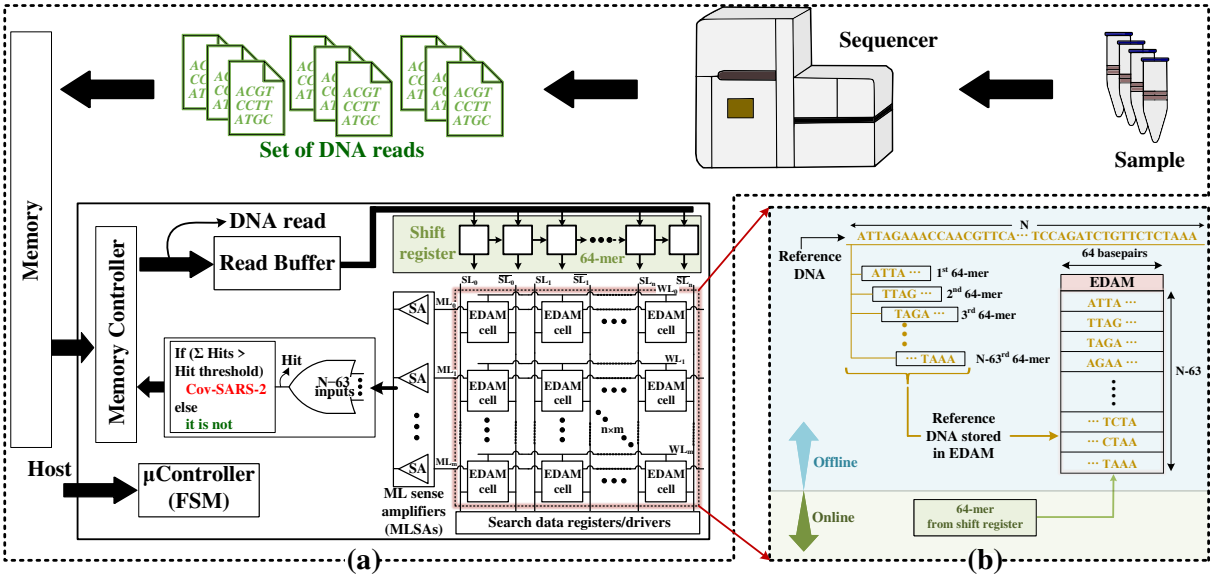

Figure 9: (a) EDAM as a part of a virus detection platform; (b) The offline construction of the reference DNA database in EDAM (top) and the online virus detection operation (bottom).

reads of the known origin (i.e., the DNA reads the results of classification for which are known). By periodically processing such validation set while varying V_{ref}, V_{eval}, t_{eval}, the optimal edit distance threshold level that maximizes a certain criteria (for example, the sensitivity, subject to a certain precision constraint, or the F_1 score) can be identified and updated.

EDAM does not implement the optimal sequence alignment; however, it can be used in DNA read alignment as an index accelerator or a prealignment filter.

5.2 Figures of merit for EDAM DNA classification efficiency

Fig. 10 shows the examples of true positive, true negative, false negative and false positive DNA classification results, generated by EDAM. While true results are trivial, a false negative result is received due to three consecutive insertions (such that the edit distance < 4), which EDAM is unable to tolerate in a single search. A false positive result is received in an unlikely case when a large number of replacements (> 4 in this example) accidentally match the left or right neighbors of the replaced bases.

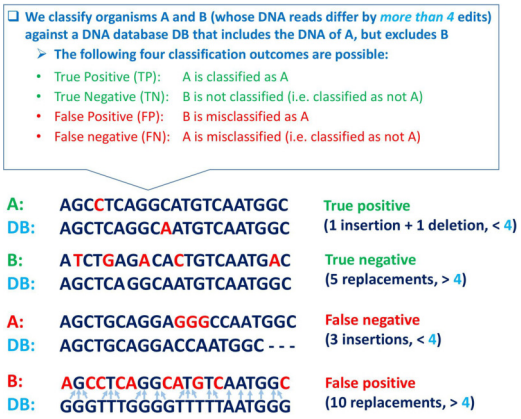

Figure 10: Examples of true positive, true negative, false negative and false positive results generated by EDAM. Edit distance threshold is 4.

The first two figures of merit, sensitivity and precision, are calculated as follows:

$$Sensitivity = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad Precision = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{2}$$

Additionally, we calculate F_1 score which is a harmonic mean of sensitivity and precision, as follows:

$$\text{F}_1 = \frac{2 \times Sensitivity \times Precision}{Sensitivity + Precision} \tag{3}$$

We use arguably the most popular state-of-the-art classification tool Kraken2 [71] as a baseline reference for the virus DNA detection and classification efficiency comparison. We use the normalized F_1 score as an additional figure of merit in the EDAM comparative evaluation:

$$\text{Normalized F}_1 = \frac{F_1(\text{EDAM})}{F_1(\text{Kraken2})} \tag{4}$$

5.3 DNA classification evaluation methodology and results

All DNA sequences in our evaluation are downloaded from NCBI online data sets [51]. We evaluate the EDAM DNA classification efficiency by conducting two different experiments, one using synthetic virus DNA reads and the other one performing DNA classification of raw DNA reads produced by an industrial sequencer.

In the first experiment, we apply EDAM to virus detection, which is the same as classifying DNA reads to a specific virus genome. In particular, we attempt to detect the coronavirus SARS-CoV-2 and its variants (alpha - B.1.1.7, beta - B.1.351 and gamma - P.1) in a synthetic metagenomic sample, containing DNA reads of the above listed organisms, as well as the DNA of several other organisms:

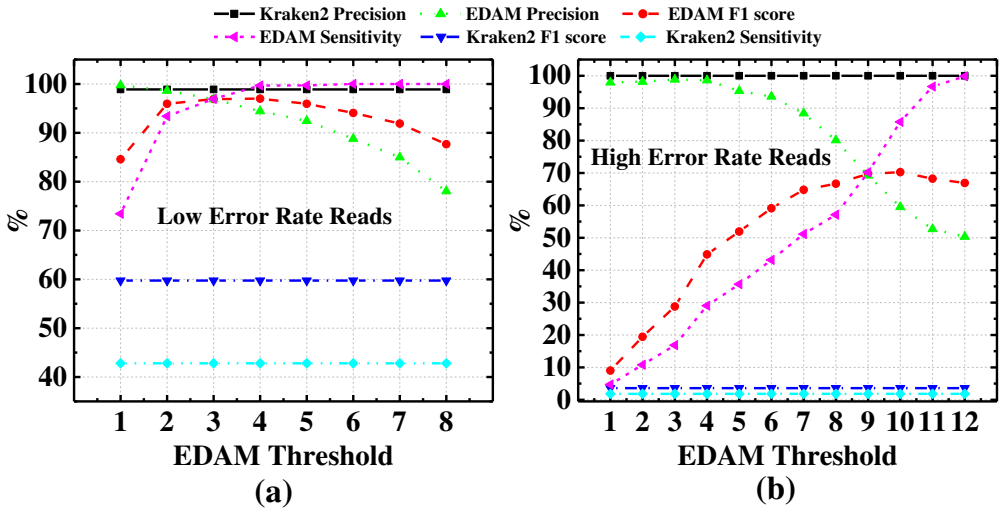

Figure 11: Sensitivity, precision and F_1 score for synthetic (a) low error rate and (b) high error rate reads.

SARS-CoV-1, MERS-CoV, Coronavirus HKU1 and Human papillomavirus (HPV) 14. The 64-base-long DNA reads were extracted from random positions in the DNA sequences of each of these organisms. Consecutively, sequencing errors (insertions, deletions and replacements) were randomly injected. Using these reads, a metagenomic dataset was created with the following two error rates:

(1) Low error rate reads: Replacement = 3.6%, insertion = 0.2%, deletion = 0.2%, simulating the low error rate of the second generation DNA sequencers [28].
(2) High error rate reads: Replacement = 1%, insertion = 7%, deletion = 7%, simulating the high error rate of the third generation DNA sequencers [28].

Each DNA read (a 64-mer) in the metagenomic sample is searched in the EDAM, which stores the SARS-CoV-2 DNA as a reference database. The error-injected reads of SARS-CoV-2 and its variants that match and mismatch in the EDAM constitute true positive and false negative results, respectively. DNA reads of the other organisms that match and mismatch in EDAM, constitute false positive and true negative results, respectively.

To compare EDAM classification efficiency and performance with those of Kraken2, a Kraken2 database containing only SARS-CoV-2 DNA, was created. In other words, Kraken2 was operated in a detection rather than a full classification mode. Kraken2 was applied to our synthetic metagenomic dataset using 64-mers.

The sensitivity, precision and F_1 score results of the first experiment (as the function of the edit distance threshold) are presented in Fig. 11. The classification efficiency of Kraken2 does not depend on EDAM edit distance threshold; hence, its sensitivity, precision and F_1 score are presented by horizontal lines. We observe that EDAM sensitivity grows with the increasing edit distance threshold for both low and high error rate read classification runs. At the same time, EDAM precision diminishes (due to the growing number of false positive matches) for both runs. There is a point where the drop in precision begins to outpace the growth of sensitivity. This point marks the optimal F_1 score which occurs at different edit distance threshold values (4 for low error rate and 9 for high error rate, respectively).

The normalized F_1 score results of the first experiment are presented in Fig. 12. While EDAM outperforms Kraken2 for both low and high error rate DNA reads, the normalized F_1 score is much higher for high error rate reads. For low error reads, EDAM outperforms Kraken2 by approximately 1.42×–1.65×. This figure grows to 2.5×–19.55× for high error reads. Kraken2 fails to correctly classify certain SARS-CoV-2 reads due to sequencing errors, since it performs exact rather than approximate search. In contrast, EDAM correctly classifies more reads due to its ability to tolerate edits caused by sequencing errors.

In the second experiment, we perform classification of raw DNA reads. Specifically, we classify high quality long reads produced by PacBio sequencers RS, RS II and Sequel II (from Sequence Read Archive at NCBI [51]). Similar to first experiment, a metagenomic DNA sample was created, comprising raw reads from the following organisms: Human Alphaherpesvirus 2, Pandoravirus Salinus and SARS-CoV-2. In this experiment, a Human Alphaherpesvirus 2 virus DNA (NCBI [51]) was used as a reference (target) virus DNA. Hence, the raw reads of Human Alphaherpesvirus 2 are expected to match, i.e., yield positive results, while the rest of the reads should ideally mismatch (i.e., produce negative results). The reference database was produced by extracting 64-mers of the clean Human Alphaherpesvirus 2 DNA and storing them in EDAM.

EDAM sensitivity, precision and F_1 score results of the second experiment are presented in Fig. 13. Similar to the first experiment outcome, EDAM sensitivity grows with the increasing edit distance threshold, while its precision decreases. EDAM F_1 score reaches its maximum at the edit distance threshold of 6. As the edit distance threshold continues to grow, the sensitivity rise is increasingly outpaced by the precision drop, leading to the F_1 score reduction.

The normalized F_1 score results of the second experiment are shown at Fig. 14. The overall behavior is similar to that of the first experiment with low error rate reads. EDAM outperforms Kraken2 by 1.31× to 1.7×. EDAM correctly classifies more reads than Kraken2. However, starting at the edit distance threshold of 6, EDAM incorrectly classifies an increasingly growing numbers of

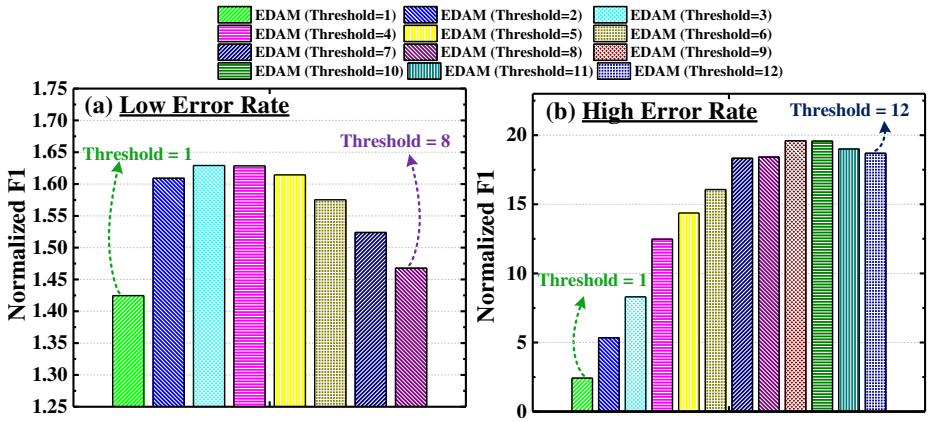

Figure 12: (a) EDAM normalized F1 score for synthetic (a) low error rate reads, (b) high error rate reads.

Pandoravirus Salinus and SARS-CoV-2 reads as Human Alphaherpesvirus 2 (generating false positive results).

Based on the results of two experiments, we conclude as follows:

(1) EDAM strongly outperforms Kraken2, and the factor grows with the DNA read error rate, reaching 19.55× for reads with 15% error rate.

(2) F_1 score has an optimum point, whose position depends on the error rate. For the low edit distance threshold values, the growing EDAM sensitivity outpaces the decreasing precision, leading to the F_1 score growth. However, as the edit distance threshold increases, the drop in precision surpasses the rise in sensitivity, causing the F_1 score to diminish.

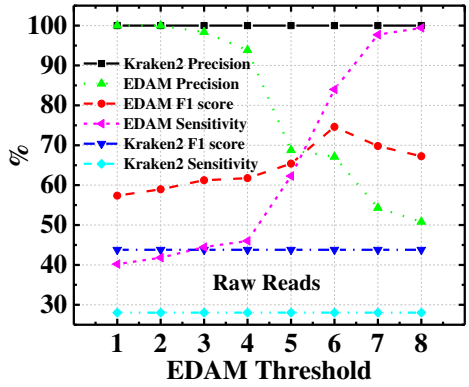

Figure 13: Sensitivity, precision and F_1 score for PacBio reads

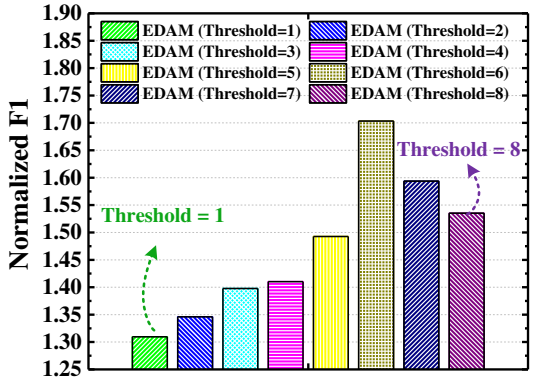

Figure 14: EDAM normalized F1 score for PacBio reads

Table 1: EDAM vs. Kraken2 comparison summary

Figure of Merit	Experiment	Result
Max normalized F1 score	High Error Rate Reads	19.55
	Low Error Rate Reads	1.65
	PacBio Reads	1.70
Min normalized F1 score	High Error Rate Reads	2.51
	Low Error Rate Reads	1.42
	PacBio Reads	1.31
Speedup	High Error Rate Reads	1080.71
	Low Error Rate Reads	1085.28
	PacBio Reads	1621.06

Normalized F_1 score is the EDAM F_1 score to Kraken2 F_1 score ratio. Kraken2 is run on i9-10900X CPU operating at 3.70GHz with 32GB of 2,133MHz DDR4 main memory

Table 2: EDAM area, power and timing.

Technology	65-nm
Nominal supply voltage	1.2 V
Cell area	33.37 um^2
Number of transistors per cell	42
EDAM word size	64 cells*
Search time	1.5 ns
Average power per cell	0.94 µW

*each EDAM cell contains 2 SRAM, i.e., 128-bit

The EDAM vs. Kraken2 normalized F_1 score results are summarized in Table 1.

5.4 Speedup vs. Kraken2

We run Kraken2 on Intel Core i9-10900X CPU operating at 3.70GHz with 32GB of 2,133MHz DDR4 main memory. Kraken2 classification throughput figures for high error rate reads, low rate error reads and raw PacBio reads are $2.37\frac{\text{Giga basepairs}}{\text{Minute}}$, $2.36\frac{\text{Giga basepairs}}{\text{Minute}}$ and $1.58\frac{\text{Giga basepairs}}{\text{Minute}}$, respectively The average Kraken2 classification throughput is $2.11\frac{\text{Giga basepairs}}{\text{Minute}}$.

EDAM processes one k-mer per cycle, hence its classification throughput is $f_{op} \times k$, where f_{op} is the EDAM operating frequency

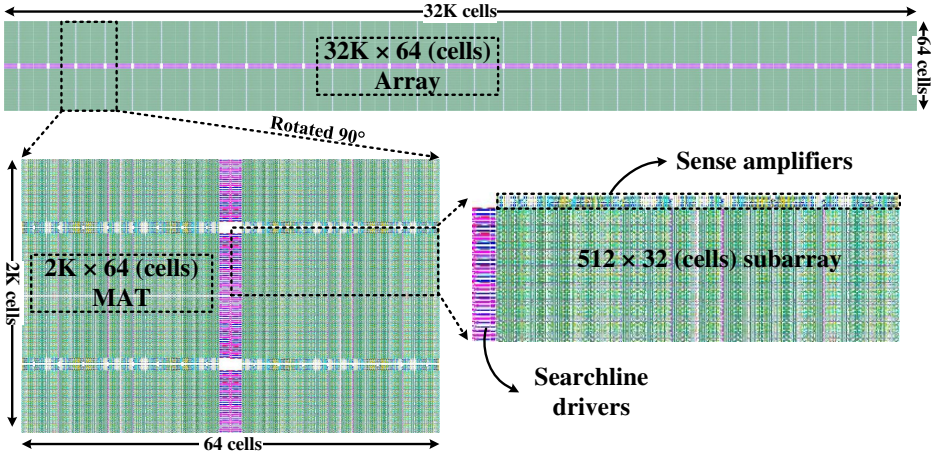

Figure 15: Layout of the $32K \times 64(cells)$ EDAM used as DNA classifier.

and k is the k-mer size. For $f_{op} = 667$ MHz and $k = 64$, the classification throughput is $2,561 \frac{\text{Giga basepairs}}{\text{Minute}}$, achieving an average speedup of 1,214× over Kraken2. The individual speedup figures are presented in Table 1.

5.5 Area and power evaluation

The EDAM array was designed using a commercial 65 nm CMOS process, simulated and manually laid out using Cadence Spectre and Virtuoso. The main area and power consumption figures are summarized in Table 2. EDAM operates at a nominal supply voltage of 1.2 V and consumes an average power 0.94 µW per cell. Our design achieves edit distance-tolerant operations at the expense of larger cell footprint (42 transistors per EDAM cell). Fig. 15 presents the layout of a 32K×64 (cells) EDAM array (including its peripheral circuitry) employed as a DNA detector and classifier. Its silicon area is 68 mm^2 and its power consumption is 1.92 W[1].

6 CONCLUSION

In this paper, we present EDAM, a novel Edit Distance tolerant Approximate Matching content addressable memory architecture. EDAM enables a parallel search with a user configurable edit distance tolerance. EDAM cell is based on 6T-SRAM bitcells with additional comparison logic that enables the edit distance tolerance. The EDAM cell also comprises an M_{eval} transistor that controls the pace of the matchline discharge. The conductivity of the M_{eval} transistor is configured by setting $V_{\text{eval}} = V_{\text{DD}}$ or $V_{\text{eval}} < V_{\text{DD}}$, thus enabling exact or approximate search, respectively. EDAM was designed using a commercial 65 nm 1.2 V CMOS technology, and evaluated through extensive Monte Carlo simulations at different process corners.

A comprehensive design space analysis shows that EDAM successfully functions in a wide range of edit distance threshold values, while maintaining a very high sensitivity and specificity. The effects of process variations can be successfully mitigated by dynamically adjusting several EDAM user-defined parameters.

EDAM enables an efficient genomic surveillance using its ability to classify viruses and other pathogen DNA quickly and with great sensitivity and precision. We show that depending on the sequencing error rate, EDAM provides 1.31× to 19.55× higher F_1 score compared to arguably the most popular state of the art DNA classifier Kraken2, while outperforming it by 1,214× on average.

[1] Corresponding to the worst case when the ML of all EDAM rows discharge completely.

REFERENCES

[1] Pablo Acera Mateos, Renzo F Balboa, Simon Easteal, Eduardo Eyras, and Hardip R Patel. 2021. PACIFIC: a lightweight deep-learning classifier of SARS-CoV-2 and co-infecting RNA viruses. *Scientific reports* 11, 1 (2021), 1–14.

[2] Amit Agarwal, Steven Hsu, Sanu Mathew, Mark Anders, Himanshu Kaul, Farhana Sheikh, and Ram Krishnamurthy. 2011. A 128×128b high-speed wide-and match-line content addressable memory in 32nm CMOS. In *2011 Proceedings of the ESSCIRC (ESSCIRC)*. 83–86. https://doi.org/10.1109/ESSCIRC.2011.6044920

[3] Mustafa Ali, Amogh Agrawal, and Kaushik Roy. 2020. RAMANN: In-SRAM Differentiable Memory Computations for Memory-Augmented Neural Networks. In *Proceedings of the ACM/IEEE International Symposium on Low Power Electronics and Design* (Boston, Massachusetts) *(ISLPED '20)*. Association for Computing Machinery, New York, NY, USA, 61–66. https://doi.org/10.1145/3370748.3406574

[4] Stephen F Altschul, Warren Gish, Webb Miller, Eugene W Myers, and David J Lipman. 1990. Basic local alignment search tool. *Journal of molecular biology* 215, 3 (1990), 403–410.

[5] Stephen Armstrong. 2020. Covid-19: Government buried negative data on its favoured antibody test. *BMJ: British Medical Journal (Online)* 371 (2020).

[6] Igor Arsovski, Akhilesh Patil, Robert M. Houle, Michael T. Fragano, Ramon Rodriguez, Raymond Kim, and Van Butler. 2018. 1.4Gsearch/s 2-Mb/mm2 TCAM Using Two-Phase-Pre-Charge ML Sensing and Power-Grid Pre-Conditioning to Reduce Ldi/dt Power-Supply Noise by 50%. *IEEE Journal of Solid-State Circuits* 53, 1 (2018), 155–163. https://doi.org/10.1109/JSSC.2017.2739178

[7] I Made Artika, Ageng Wiyatno, and Chairin Nisa Ma'roef. 2020. Pathogenic viruses: Molecular detection and characterization. *Infection, Genetics and Evolution* 81 (2020), 104215.

[8] Vikas Bansal and Christina Boucher. 2019. Sequencing technologies and analyses: where have we been and where are we going?

[9] Andreas V Bechtolsheim and David R Cheriton. 2002. Access control list processing in hardware. US Patent 6,377,577.

[10] Joshua S. Bloom, Laila Sathe, Chetan Munugala, Eric M. Jones, Molly Gasperini, Nathan B. Lubock, Fauna Yarza, Erin M. Thompson, Kyle M. Kovary, Jimin Park, Dawn Marquette, Stephania Kay, Mark Lucas, TreQuan Love, A. Sina Booeshaghi, Oliver F. Brandenberg, Longhua Guo, James Boocock, Myles Hochman, Scott W. Simpkins, Isabella Lin, Nathan LaPierre, Duke Hong, Yi Zhang, Gabriel Oland, Bianca Judy Choe, Sukantha Chandrasekaran, Evann E. Hilt, Manish J. Butte, Robert Damoiseaux, Clifford Kravit, Aaron R. Cooper, Yi Yin, Lior Pachter, Omai B. Garner, Jonathan Flint, Eleazar Eskin, Chongyuan Luo, Sriram Kosuri, Leonid Kruglyak, and Valerie A. Arboleda. 2020. Swab-Seq: A high-throughput platform for massively scaled up SARS-CoV-2 testing. (Aug. 2020). https://doi.org/10.1101/2020.08.04.20167874

[11] Arthur Brady and Steven L Salzberg. 2009. Phymm and PhymmBL: metagenomic phylogenetic classification with interpolated Markov models. *Nature methods* 6, 9 (2009), 673–676.

[12] Trong Tu Bui and Tadashi Shibata. 2010. A Low-Power Associative Processor with the R-th Nearest-Match Hamming-Distance Search Engine Employing Time-Domain Techniques. In *2010 Fifth IEEE International Symposium on Electronic Design, Test Applications*. 54–57. https://doi.org/10.1109/DELTA.2010.37

[13] Oscar Castañeda, Maria Bobbett, Alexandra Gallyas-Sanhueza, and Christoph Studer. 2019. PPAC: A versatile in-memory accelerator for matrix-vector-product-like operations. In *2019 IEEE 30th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*, Vol. 2160. IEEE, 149–156.

[14] Yun-Sheng Chan, Po-Tsang Huang, Shang-Lin Wu, Sheng-Chi Lung, Wei-Chang Wang, Wei Hwang, and Ching-Te Chuang. 2018. 0.4V Reconfigurable Near-Threshold TCAM in 28nm High-k Metal-Gate CMOS Process. In *2018 31st IEEE International System-on-Chip Conference (SOCC)*. 272–277. https://doi.org/10.1109/SOCC.2018.8618562

[15] Yen-Jen Chang and Yuan-Hong Liao. 2008. Hybrid-Type CAM Design for Both Power and Performance Efficiency. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 16, 8 (2008), 965–974. https://doi.org/10.1109/TVLSI.2008.2000595

[16] H Jonathan Chao. 2002. Next generation routers. *Proc. IEEE* 90, 9 (2002), 1518–1558.

[17] Kuo-Hsing Cheng, Chia-Hung Wei, and Yu-Wen Chen. 2003. Design of low-power content-addressable memory cell. In *2003 46th Midwest Symposium on Circuits and Systems*, Vol. 3. 1447–1450 Vol. 3. https://doi.org/10.1109/MWSCAS.2003.1562568

[18] Peter JA Cock, Christopher J Fields, Naohisa Goto, Michael L Heuer, and Peter M Rice. 2010. The Sanger FASTQ file format for sequences with quality scores, and the Solexa/Illumina FASTQ variants. *Nucleic acids research* 38, 6 (2010), 1767–1771.

[19] Carlo C Del Mundo, Vincent T Lee, Luis Ceze, and Mark Oskin. 2015. Ncam: Near-data processing for nearest neighbor search. In *Proceedings of the 2015 International Symposium on Memory Systems*. 274–275.

[20] Anh-Tuan Do, Shoushun Chen, Zhi-Hui Kong, and Kiat Seng Yeo. 2013. A High Speed Low Power CAM With a Parity Bit and Power-Gated ML Sensing. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 21, 1 (2013), 151–156. https://doi.org/10.1109/TVLSI.2011.2178276

[21] Anh Tuan Do, Chun Yin, Kiat Seng Yeo, and Tony Tae-Hyoung Kim. 2013. Design of a power-efficient CAM using automated background checking scheme for small match line swing. In *2013 Proceedings of the ESSCIRC (ESSCIRC)*. 209–212. https://doi.org/10.1109/ESSCIRC.2013.6649109

[22] Qing Dong, Supreet Jeloka, Mehdi Saligane, Yejoong Kim, Masaru Kawaminami, Akihiko Harada, Satoru Miyoshi, Makoto Yasuda, David Blaauw, and Dennis Sylvester. 2018. A 4 + 2T SRAM for Searching and In-Memory Computing With 0.3-V V_{DDmin}. *IEEE Journal of Solid-State Circuits* 53, 4 (2018), 1006–1015. https://doi.org/10.1109/JSSC.2017.2776309

[23] Marc Gregory Dumont, Claudia Lüke, Yongcui Deng, and Peter Frenzel. 2014. Classification of pmoA amplicon pyrosequences using BLAST and the lowest common ancestor method in MEGAN. *Frontiers in Microbiology* 5 (2014), 34.

[24] Tim Dunn, Harisankar Sadasivan, Jack Wadden, Kush Goliya, Kuan-Yu Chen, David Blaauw, Reetuparna Das, and Satish Narayanasamy. 2021. SquiggleFilter: An Accelerator for Portable Virus Detection. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*. 535–549.

[25] Wanling Gao, Yuqing Zhu, Zhen Jia, Chunjie Luo, Lei Wang, Zhiguo Li, Jianfeng Zhan, Yong Qi, Yongqiang He, Shiming Gong, Xiaona Li, Shujie Zhang, and Bizhu Qiu. 2013. BigDataBench: a big data benchmark suite from web search engines. *arXiv preprint arXiv:1307.0320* (2013).

[26] Lilit Garibyan and Nidhi Avashia. 2013. Research techniques made simple: polymerase chain reaction (PCR). *The Journal of investigative dermatology* 133, 3 (2013), e6.

[27] Esteban Garzón, Roman Golman, Zuher Jahshan, Robert Hanhan, Natan Vinshtok-Melnik, Marco Lanuzza, Adam Teman, and Leonid Yavits. 2022. Hamming Distance Tolerant Content-Addressable Memory (HD-CAM) for DNA Classification. *IEEE Access* 10 (2022), 28080–28093.

[28] Sara Goodwin, John D McPherson, and W Richard McCombie. 2016. Coming of age: ten years of next-generation sequencing technologies. *Nature Reviews Genetics* 17, 6 (2016), 333–351.

[29] Sheikh Wasmir Hussain, Telajala Venkata Mahendra, Sandeep Mishra, and Anup Dandapat. 2018. Match-Line Division and Control to Reduce Power Dissipation in Content Addressable Memory. *IEEE Transactions on Consumer Electronics* 64, 3 (2018), 301–309. https://doi.org/10.1109/TCE.2018.2859623

[30] Illumina. 2021. Illumina - DNA Sequencing. https://www.illumina.com/techniques/sequencing/dna-sequencing.html

[31] Mohsen Imani, Yeseong Kim, Abbas Rahimi, and Tajana Rosing. 2016. ACAM: Approximate Computing Based on Adaptive Associative Memory with Online Learning. In *Proceedings of the 2016 International Symposium on Low Power Electronics and Design* (San Francisco Airport, CA, USA) *(ISLPED '16)*. Association for Computing Machinery, New York, NY, USA, 162–167. https://doi.org/10.1145/2934583.2934595

[32] Mohsen Imani, Abbas Rahimi, Deqian Kong, Tajana Rosing, and Jan M Rabaey. 2017. Exploring hyperdimensional associative memory. In *2017 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 445–456.

[33] D Jothi and R Sivakumar. 2018. Design and analysis of power efficient binary content addressable memory (PEBCAM) core cells. *Circuits, Systems, and Signal Processing* 37, 4 (2018), 1422–1451.

[34] Roman Kaplan, Leonid Yavits, and Ran Ginosar. 2018. PRINS: Processing-in-Storage Acceleration of Machine Learning. *IEEE Transactions on Nanotechnology* 17, 5 (2018), 889–896. https://doi.org/10.1109/TNANO.2018.2799872

[35] Roman Kaplan, Leonid Yavits, and Ran Ginosar. 2018. RASSA: resistive prealignment accelerator for approximate DNA long read mapping. *IEEE Micro* 39, 4 (2018), 44–54.

[36] Roman Kaplan, Leonid Yavits, Ran Ginosar, and Uri Weiser. 2017. A Resistive CAM Processing-in-Storage Architecture for DNA Sequence Alignment. *IEEE Micro* 37, 4 (2017), 20–28. https://doi.org/10.1109/MM.2017.3211121

[37] Roman Kaplan, Leonid Yavits, and Ran Ginosasr. 2020. BioSEAL: In-Memory Biological Sequence Alignment Accelerator for Large-Scale Genomic Data. In *Proceedings of the 13th ACM International Systems and Storage Conference*. 36–48.

[38] S Karen Khatamifard, Zamshed Chowdhury, Nakul Pande, Meisam Razaviyayn, Chris Kim, and Ulya R Karpuzcu. 2017. Read Mapping Near Non-Volatile Memory. *arXiv preprint arXiv:1709.02381* (2017).

[39] Jeremie S Kim, Damla Senol Cali, Hongyi Xin, Donghyuk Lee, Saugata Ghose, Mohammed Alser, Hasan Hassan, Oguz Ergin, Can Alkan, and Onur Mutlu. 2018. GRIM-Filter: Fast seed location filtering in DNA read mapping using processing-in-memory technologies. *BMC genomics* 19, 2 (2018), 23–40.

[40] T. Kobayashi, K. Nogami, T. Shirotori, Y. Fujimoto, and O. Watanabe. 1992. A current-mode latch sense amplifier and a static power saving input buffer for low-power architecture. In *1992 Symposium on VLSI Circuits Digest of Technical Papers*. 28–29. https://doi.org/10.1109/VLSIC.1992.229252

[41] Sriram C. Krishnan, Rina Panigrahy, and Sunil Parthasarathy. 2009. Error-Correcting Codes for Ternary Content Addressable Memories. *IEEE Trans. Comput.* 58, 2 (2009), 275–279. https://doi.org/10.1109/TC.2008.179

[42] Ben Langmead. 2010. Aligning short sequencing reads with Bowtie. *Current protocols in bioinformatics* 32, 1 (2010), 11–7.

[43] Heng Li. 2013. Aligning sequence reads, clone sequences and assembly contigs with BWA-MEM. *arXiv preprint arXiv:1303.3997* (2013).

[44] Heng Li. 2016. Minimap and miniasm: fast mapping and de novo assembly for noisy long sequences. *Bioinformatics* 32, 14 (2016), 2103–2110.

[45] Heng Li. 2018. Minimap2: pairwise alignment for nucleotide sequences. *Bioinformatics* 34, 18 (2018), 3094–3100.

[46] Wenming Li, Xiaochun Ye, Da Wang, Hao Zhang, Zhimin Tang, Dongrui Fan, and Ninghui Sun. 2019. PIM-WEAVER: A High Energy-efficient, General-purpose Acceleration Architecture for String Operations in Big Data Processing. *Sustainable Computing: Informatics and Systems* 21 (2019), 129–142.

[47] Bo Liu, Theodore Gibbons, Mohammad Ghodsi, Todd Treangen, and Mihai Pop. 2011. Accurate and fast estimation of taxonomic profiles from metagenomic shotgun sequences. *Genome biology* 12, 1 (2011), 1–27.

[48] Guillaume Marçais, Dan DeBlasio, Prashant Pandey, and Carl Kingsford. 2019. Locality-sensitive hashing for the edit distance. *Bioinformatics* 35, 14 (2019), i127–i135.

[49] H.J. Mattausch, T. Gyohten, Y. Soda, and T. Koide. 2002. Compact associative-memory architecture with fully parallel search capability for the minimum Hamming distance. *IEEE Journal of Solid-State Circuits* 37, 2 (2002), 218–227. https://doi.org/10.1109/4.982428

[50] Sandeep Mishra, Telajala Venkata Mahendra, and Anup Dandapat. 2016. A 9-T 833-MHz 1.72-fJ/Bit/Search Quasi-Static Ternary Fully Associative Cache Tag With Selective Matchline Evaluation for Wire Speed Applications. *IEEE Transactions on Circuits and Systems I: Regular Papers* 63, 11 (2016), 1910–1920. https://doi.org/10.1109/TCSI.2016.2592182

[51] NCBI. 2021. Bethesda (MD): National Library of Medicine (US), National Center for Biotechnology Information. https://www.ncbi.nlm.nih.gov/

[52] Patrick Ng. 2017. dna2vec: Consistent vector representations of variable-length k-mers. *arXiv preprint arXiv:1701.06279* (2017).

[53] Kai Ni, Xunzhao Yin, Ann Franchesca Laguna, Siddharth Joshi, Stefan Dünkel, Martin Trentzsch, Johannes Müller, Sven Beyer, Michael Niemier, Xiaobo Sharon Hu, and Suman Datta. 2019. Ferroelectric ternary content-addressable memory for one-shot learning. *Nature Electronics* 2, 11 (2019), 521–529.

[54] ONT. 2021. MinION – Portable real-time devices for DNA and RNA sequencing. https://nanoporetech.com/products/minion

[55] Agogo E. Otu A. and Ebenso B. 2021. Africa needs more genome sequencing to tackle new variants of SARS-CoV-2. *Nature Medicine* (2021), 744–745.

[56] Rachid Ounit and Stefano Lonardi. 2016. Higher classification sensitivity of short metagenomic reads with CLARK-S. *Bioinformatics* 32, 24 (2016), 3823–3825.

[57] Rachid Ounit, Steve Wanamaker, Timothy J Close, and Stefano Lonardi. 2015. CLARK: fast and accurate classification of metagenomic and genomic sequences using discriminative k-mers. *BMC genomics* 16, 1 (2015), 1–13.

[58] Kostas Pagiamtzis, Navid Azizi, and Farid N. Najm. 2006. A Soft-Error Tolerant Content-Addressable Memory (CAM) Using An Error-Correcting-Match Scheme. In *IEEE Custom Integrated Circuits Conference 2006*. 301–304. https://doi.org/10.1109/CICC.2006.320887

[59] K. Pagiamtzis and A. Sheikholeslami. 2006. Content-addressable memory (CAM) circuits and architectures: a tutorial and survey. *IEEE Journal of Solid-State Circuits* 41, 3 (2006), 712–727. https://doi.org/10.1109/JSSC.2005.864128

[60] K. Prasanth, M. Ramireddy, T. Keerthi priya, and S. Ravindrakumar. 2019. High Speed, Low Matchline Voltage Swing and Search Line Activity TCAM Cell Array Design in 14 nm FinFET Technology. In *Lecture Notes in Electrical Engineering*. Springer Singapore, 465–473. https://doi.org/10.1007/978-981-13-8942-9_38

[61] Abbas Rahimi, Amirali Ghofrani, Kwang-Ting Cheng, Luca Benini, and Rajesh K. Gupta. 2015. Approximate associative memristive memory for energy-efficient GPUs. In *2015 Design, Automation Test in Europe Conference Exhibition (DATE)*. 1497–1502. https://doi.org/10.7873/DATE.2015.0579

[62] Akshay Krishna Ramanathan, Srivatsa Srinivasa Rangachar, Je-Min Hung, Chun-Ying Lee, Cheng-Xin Xue, Sheng-Po Huang, Fu-Kuo Hsueh, Chang-Hong Shen, Jia-Min Shieh, Wen-Kuan Yeh, Mon-Shu Ho, Hariram Thirucherai Govindarajan, Jack Sampson, Meng-Fan Chang, and Vijaykrishnan Narayanan. 2020. Monolithic 3D+-IC Based Massively Parallel Compute-in-Memory Macro for Accelerating Database and Machine Learning Primitives. In *2020 IEEE International Electron Devices Meeting (IEDM)*. 28.5.1–28.5.4. https://doi.org/10.1109/IEDM13553.2020.9372111

[63] M Sadegh Riazi, Mohammad Samragh, and Farinaz Koushanfar. 2017. Camsure: Secure content-addressable memory for approximate search. *ACM Transactions on Embedded Computing Systems (TECS)* 16, 5s (2017), 1–20.

[64] Meysam Roodi and Andreas Moshovos. 2018. Gene sequencing: where time goes. In *2018 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 84–85.

[65] Gail Rosen, Elaine Garbarine, Diamantino Caseiro, Robi Polikar, and Bahrad Sokhansanj. 2008. Metagenome Fragment Classification Using N-Mer Frequency Profiles. *Advances in bioinformatics* 2008 (2008).

[66] Abigail Sawyer, Tristan Free, and Joseph Martin. 2021. Metagenomics: preventing future pandemics.

[67] Divya Sethi, Manjit Kaur, and Gurmohan Singh. 2017. Design and performance analysis of a CNFET-based TCAM cell with dual-chirality selection. *Journal of Computational Electronics* 16, 1 (2017), 106–114.

[68] Brajesh K Singh, Pankaj Trivedi, Eleonora Egidi, Catriona A Macdonald, and Manuel Delgado-Baquerizo. 2020. Crop microbiome and sustainable agriculture. *Nature Reviews Microbiology* 18, 11 (2020), 601–602.

[69] Zachary D Stephens, Skylar Y Lee, Faraz Faghri, Roy H Campbell, Chengxiang Zhai, Miles J Efron, Ravishankar Iyer, Michael C Schatz, Saurabh Sinha, and Gene E Robinson. 2015. Big data: astronomical or genomical? *PLoS biology* 13, 7 (2015), e1002195.

[70] Mohammad MA Taha and Christof Teuscher. 2020. Approximate memristive in-memory Hamming distance circuit. *ACM Journal on Emerging Technologies in Computing Systems (JETC)* 16, 2 (2020), 1–14.

[71] Derrick E Wood, Jennifer Lu, and Ben Langmead. 2019. Improved metagenomic analysis with Kraken 2. *Genome biology* 20, 1 (2019), 1–13.

[72] Derrick E Wood and Steven L Salzberg. 2014. Kraken: ultrafast metagenomic sequence classification using exact alignments. *Genome biology* 15, 3 (2014), 1–12.

[73] Leonid Yavits, Roman Kaplan, and Ran Ginosar. 2021. GIRAF: General Purpose In-Storage Resistive Associative Framework. *IEEE Transactions on Parallel and Distributed Systems* (2021), 1–1. https://doi.org/10.1109/TPDS.2021.3065448

[74] Leonid Yavits, Shahar Kvatinsky, Amir Morad, and Ran Ginosar. 2015. Resistive Associative Processor. *IEEE Computer Architecture Letters* 14, 2 (2015), 148–151. https://doi.org/10.1109/LCA.2014.2374597

[75] Leonid Yavits, Amir Morad, and Ran Ginosar. 2015. Computer Architecture with Associative Processor Replacing Last-Level Cache and SIMD Accelerator. *IEEE Trans. Comput.* 64, 2 (2015), 368–381. https://doi.org/10.1109/TC.2013.220

[76] Mohammed Zackriya V and Harish M. Kittur. 2016. Precharge-Free, Low-Power Content-Addressable Memory. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 24, 8 (2016), 2614–2621. https://doi.org/10.1109/TVLSI.2016.2518219

[77] Xuan Zhu, Xuejun Yang, Chunqing Wu, Junjie Wu, and Xun Yi. 2013. Hamming network circuits based on CMOS/memristor hybrid design. *IEICE Electronics Express* (2013), 10–20130404.